Լաբորատոր աշխատանք3

Ապրանքների պահեստ

Աշխատանքի նպատակը

- Սովորել օգտագործել կառուցվածքների զանգվածներ (`struct` զանգվածներ) C++ լեզվում:
- Յուրացնել հղումով աշխատող ֆունկցիաների ստեղծման և կիրառման սկզբունքները տվյալների փոփոխման համար:
- Կիրառել պայմանական օպերատորներ և մշակել թվային տվյալներ (սահմանափակումների ստուգումներ, արժեքների վերահաշվարկում):

Աշխատանքի պահանջը

- Ստեղծել `Product` կառուցվածք, որը պետք է պարունակի հետևյալ դաշտերը՝ կոդ (`sku`), անվանում (`name`), գին (`unitPrice`) և քանակ (`quantity`):
- Հաշվել յուրաքանչյուր ապրանքի ընդհանուր արժեքը և գտնել սահմանված գումարից թանկ պահեստային պաշարները:
- 5-ից պակաս քանակ ունեցող ապրանքների գինը ավելացնել 3% տոկոսով՝ հղումով աշխատող ֆունկցիայի միջոցով:
- **Պարտադիր կիրառել՝** կառուցվածքների զանգված, առնվազն երկու ֆունկցիա, որոնցում իրականացվում է որոնում կամ դասակարգում:
- **Արդյունք՝** Ծրագիրը պետք է արտածի մշակված գրառումները և խնդրում պահանջված ամփոփ արդյունքը:

Տեսական մաս

Կառուցվածքային տիպերը (`struct`) C++-ում թույլ են տալիս միավորել տարբեր տիպերի տվյալներ մեկ ընդհանուր տրամաբանական միավորի մեջ:

1. **Կառուցվածքների զանգվածներ:** Երբ անհրաժեշտ է պահպանել և մշակել նույն տիպի բազմաթիվ օբյեկտներ (օրինակ՝ պահեստի ապրանքների ցանկ), օգտագործվում են `struct` տիպի զանգվածներ կամ `std::vector<Product>` կոնտեյներներ:
2. **Ֆունկցիաներ հղումով (Pass by Reference):** Երբ ֆունկցիային անհրաժեշտ է փոփոխել փոխանցվող փոփոխականի կամ զանգվածի տարրի արժեքը հիշողության մեջ առանց պատճեն ստեղծելու, օգտագործվում է հղումը (`&` նշանը):
3. **Տվյալների զտում և որոնում:** Պահեստային համակարգերում հաճախակի կիրառվում են պայմանական ստուգումներ՝ որոշակի չափանիշներին (օրինակ՝ նվազագույն քանակին կամ սահմանային գնին) բավարարող ապրանքներն առանձնացնելու և դրանց պարամետրերը փոփոխելու համար:

```
#include <iostream>

#include <string>

#include <vector>

#include <iomanip>

using namespace std;

// Product կառուցվածքը

struct Product {

    string sku;

    string name;

    double unitPrice;

    int quantity;

};

// Ֆունկցիա 1. 5-ից պակաս քանակ ունեցող ապրանքների գինը 3%-ով ավելացնելու համար (հղումով աշխատող ֆունկցիա)

void increasePriceForLowStock(vector<Product>& products) {

    for (size_t i = 0; i < products.size(); ++i) {

        if (products[i].quantity < 5) {

            products[i].unitPrice += products[i].unitPrice * 0.03; // Գնի ավելացում 3%-ով

        }

    }

}

// Ֆունկցիա 2. Որոնել և արտածել սահմանված գումարից թանկ պահեստային պաշարները

void findExpensiveProducts(const vector<Product>& products, double thresholdLimit) {

    cout << "\n--- Ապրանքներ, որոնց միավորի գինը բարձր է " << thresholdLimit << " դրամից ---\n";

    bool found = false;

    for (size_t i = 0; i < products.size(); ++i) {

        if (products[i].unitPrice > thresholdLimit) {
```

```
            cout << "Կոդ: " << products[i].sku
                 << " | Անվանում: " << products[i].name
                 << " | Գին: " << products[i].unitPrice << " դր.\n";
            found = true;
        }
    }
    if (!found) {
        cout << "Այդպիսի ապրանքներ չեն հայտնաբերվել:\n";
    }
}
int main() {
    // Տեղադրում ենք հայերեն տառերի ճիշտ աշխատանքի կարգավորումը
    setlocale(LC_ALL, "Armenian");
    // Ստեղծում ենք ապրանքների զանգված (vector)
    vector<Product> warehouse = {
        {"SKU101", "Նոթբուք (Laptop)", 350000.0, 3},
        {"SKU102", "Մկնիկ (Mouse)", 12000.0, 10},
        {"SKU103", "Ստեղնաշար (Keyboard)", 18000.0, 4},
        {"SKU104", "Մոնիտոր (Monitor)", 90000.0, 2},
        {"SKU105", "Տպիչ (Printer)", 65000.0, 6}
    };
    cout << fixed << setprecision(2);
    cout << "=========================================================\n";
    cout << "          ԱՊՐԱՆՔՆԵՐԻ ՊԱՀԵՍՏԻ ՍԿԶԲՆԱԿԱՆ ՎԻՃԱԿ          \n";
    cout << "=========================================================\n";
    double totalWarehouseValue = 0;
```

```
    // Հաշվում ենք յուրաքանչյուր ապրանքի ընդհանուր արժեքը

    for (size_t i = 0; i < warehouse.size(); ++i) {

        double itemTotal = warehouse[i].unitPrice * warehouse[i].quantity;

        totalWarehouseValue += itemTotal;

         cout << "Կոդ: " << warehouse[i].sku

              << " | Անվանում: " << warehouse[i].name

              << " | Քանակ: " << warehouse[i].quantity

              << " | Գին: " << warehouse[i].unitPrice

              << " | Ընդհանուր արժեք: " << itemTotal << " դր.\n";

    }

    cout << "-----------------------------------------------------------\n";

     cout << "Պահեստի ընդհանուր արժեքը նախնական: " << totalWarehouseValue << " դր.\n";

    // Կիրառում ենք հղումով աշխատող ֆունկցիան՝ 5-ից պակաս քանակ ունեցողների գինը 3%-ով բարձրացնելու համար

    increasePriceForLowStock(warehouse);

    // Որոնում և արտածում ենք սահմանված գումարից թանկ ապրանքները

    findExpensiveProducts(warehouse, 50000.0);

    cout << "\n============================================================\n";

     cout << "          ՎԵՐՋՆԱԿԱՆ ՏՎՅԱԼՆԵՐԸ ՓՈՓՈԽՈՒԹՅՈՒՆՆԵՐԻՑ ՀԵՏՈ       \n";

    cout << "============================================================\n";



    double updatedWarehouseValue = 0;

    for (size_t i = 0; i < warehouse.size(); ++i) {

        double itemTotal = warehouse[i].unitPrice * warehouse[i].quantity;

        updatedWarehouseValue += itemTotal;

         cout << "Կոդ: " << warehouse[i].sku
```

```
            << " | Անվանում: " << warehouse[i].name
            << " | Քանակ: " << warehouse[i].quantity
            << " | Նոր գին: " << warehouse[i].unitPrice
            << " | Անհատական արժեք: " << itemTotal << " դր.\n";
    }
    cout << "-----------------------------------------------------\n";
    cout << "Պահեստի վերջնական ընդհանուր արժեքը: " << updatedWarehouseValue << " դր.\n";
    cout << "=====================================================\n";
    return 0;
}
```